# Application and Evaluation of Soft-Actor Critic Reinforcement Learning in Constrained Trajectory Planning for 6DOF Robotic Manipulators

Wanqing Xia
*Mechanical and Mechatronics Engineering*
*The University of Auckland*
Auckland, New Zealand
wxia612@aucklanduni.ac.nz

Yuqian Lu
*Mechanical and Mechatronics Engineering*
*The University of Auckland*
Auckland, New Zealand
yuqian.lu@auckland.ac.nz

Xun Xu
*Mechanical and Mechatronics Engineering*
*The University of Auckland*
Auckland, New Zealand
x.xu@auckland.ac.nz

2023 29th International Conference on Mechatronics and Machine Vision in Practice (M2VIP) | 979-8-3503-2562-1/23/$31.00 ©2023 IEEE | DOI: 10.1109/M2VIP58386.2023.10413400

***Abstract*—In the field of robotic manipulator operations, precise trajectory planning for the end-effector's position and orientation is crucial, especially in tasks such as grasping a bottle by its neck. This paper presents a novel approach utilizing Reinforcement Learning to address this issue. Specifically, we employed the Soft Actor-Critic and Hindsight Experience Replay algorithm to train a UR5e manipulator in a simulated environment, incorporating a unique design for state, action, and reward. Through comparative analysis with other reward function designs, we found that our trained Reinforcement Learning model generated a more efficient trajectory and achieved a significantly higher success rate. This study underscores the potential of our approach for enhancing the trajectory planning of robotic manipulator operations.**

***Index Terms*—Reinforcement Learning, Robot Control, Trajectory Planning**

## I. INTRODUCTION

Robotic manipulators are presently utilized extensively across the globe within the manufacturing industry, bolstering high production efficiency. In comparison to traditional manufacturing settings, the future-oriented smart manufacturing system necessitates increased responsiveness, flexibility, adaptability, and cost-competitiveness [1]. However, the execution of intricate robotic tasks in smart manufacturing introduces challenges pertaining to reliable 3D perception, high-level planning, and sophisticated control [2]. The implementation of a task and navigation towards the stipulated pose requires planning a brief, collision-free trajectory for the robotic manipulator. The advent of neural networks has spurred numerous research initiatives aimed at resolving the trajectory planning problem via reinforcement learning (RL).

In recent years, various deep reinforcement learning (DRL) algorithms have been proposed for solving excessive amount of robotic tasks, such as bin picking [3], door opening [4], and object stacking [5]. These algorithms typically employ vision sensing and deep neural networks to enable the model to learn high-level, multi-step tasks through a trial-and-error approach. Nonetheless, this approach may precipitate a lack of flexibility as the task parameters are embedded within the neural network, and the model is limited to solving a specific problem. Moreover, the necessity of repeatedly learning the same basic actions can be time-consuming and inefficient. An alternative solution may involve decomposing tasks into smaller, more modular units to enhance the efficiency and flexibility of learning in robotics. For instance, a grasping task can be segmented into an action sequence involving ① recognizing the object, ② analyzing the most stable grasping point and pose, ③ moving the Tool Centre Point (TCP) of the robot to the desired pose, and finally, ④ closing the gripper. Employing rapid and reliable trajectory planning algorithms for action ③ allows the DRL model to more expeditiously learn actions ① and ②. This lets us to decompose complex policies into a sequence of primitive actions that can be independently learned and reused, thus potentially increasing the efficiency and flexibility of the learning process for robotic tasks.

The primary focus of this paper is the application of reinforcement learning to solve the trajectory planning problem for a robotic manipulator, with particular emphasis on the fundamental reaching action constrained by the TCP position and orientation. Although RL has been previously deployed in trajectory planning, most approaches solely consider reaching the desired position devoid of collisions, neglecting TCP orientation. Nonetheless, it is imperative to constrain both the target location and the spatial orientation in robotic manipulation for a comprehensive spectrum of tasks. Our objective is to formulate an RL model capable of planning a trajectory and controlling the manipulator to closely achieve the strictly constrained goal TCP position and orientation for

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on November 01,2026 at 04:20:47 UTC from IEEE Xplore. Restrictions apply.

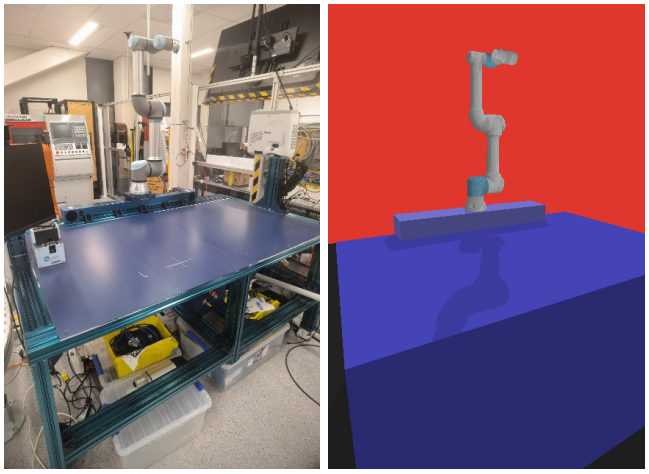


Fig. 1. The UR5e setup in our lab VS The UR5e setup in PyBullet

optimally grasping an object. We propose the application of the Soft Actor-Critic (SAC) [6] algorithm on a six degree-of-freedom manipulator and present our unique reward design to tackle this issue without specifically modifying the vanilla RL algorithm. Fig.1 depicts the simulated manipulator in a virtual environment, identical to our real robot setup, used to train the RL model. The performance of the trained model is contrasted with other models featuring different reward designs.

The remainder of this paper is organized as follows: Section II reviews relevant literature, while Section III elaborates on the reinforcement learning algorithm and our model design. Simulation results and discussions are presented in Section IV. Finally, conclusions drawn from this work and future research directions are outlined in Section V.

## II. LITERATURE REVIEW

### A. Reinforcement Learning

Although the field of manipulator trajectory planning presents several challenges, numerous researchers have applied Deep Reinforcement Learning algorithms to address this problem, yielding promising outcomes. Broadly, DRL algorithms are categorized into off-policy and on-policy methods [7].

Off-policy methods include strategies like Deep Deterministic Policy Gradient (DDPG) [8], Normalized Advantage Functions (NAF) [9], and Soft Actor-Critic [6]. These methods utilize replay buffers during algorithm training: when an action is taken, and the state transitions with reward feedback, the information regarding the action is stored in the buffer for later replay.

On-policy methods, which include algorithms such as Trust Region Policy Optimization (TRPO) [10], Natural Policy Gradient (NPG) [11], and Proximal Policy Optimization (PPO) [12], operate differently. The state, action, and reward information in on-policy methods are discarded after a single training process, resulting in these methods generally requiring many more training examples than off-policy methods.

Despite the data-hungry nature of on-policy methods, several studies have employed these algorithms for manipulator trajectory planning [13] [14]. This is mainly due to their advantages such as user-friendliness, improved stability, and robustness to sub-optimal hyperparameter settings. However, in this research, we opted for the SAC algorithm owing to its rapid training speed, superior performance and better support on continuous actions.

### B. Trajectory Planning with Constraints

The constraints concerning the goal position and orientation of the robotic manipulator's TCP have been addressed in various studies using different methods. For instance, Li et al. [15] proposed a constrained differential evolution (DE) scheme for controlling the end-effector's movements. In another study [16], the authors integrated an artificial potential field (APF) with the DDPG algorithm to define the state and reward. The APF's inclusion facilitates the reinforcement learning model in discovering the optimal solution faster. In a different approach, Lin et al. [17] combined a Long Short-Term Memory (LSTM) network with DDPG to address this problem through recurrent reinforcement learning, attaining a 91% success rate. Finally, Cao et al. [18] utilized inverse kinematics as prior knowledge to guide and accelerate the training process. Their trained model was capable of catching rotating objects. These various methodologies showcase the breadth of approaches available for tackling the challenges of constrained manipulator trajectory planning via reinforcement learning.

## III. PROBLEM FORMULATION

In this section, we formulate the constrained trajectory planning problem as a Markov Decision Process (MDP) and explain how all the Soft Actor-Critic algorithm is utilised to solve our problem. In addition, we also discuss the Hindsight Experience Replay (HER) algorithm, which we found to be highly beneficial in our experiments for accelerating our model's learning.

### A. Model Design

In reinforcement learning, the term "agent" refers to the decision-maker who interacts with the environment. In our case, the agent is the RL algorithm controlled 6-DOF (6 joints) UR5e manipulator. MDPs are used to model these sequential decision problems. An MDP can be defined by a tuple $< \mathcal{S}, \mathcal{A}, \mathcal{R}, \mathcal{P}, \gamma >$, which consists of a state set $\mathcal{S}$, an action set $\mathcal{A}$, a reward function $\mathcal{R}(s_t, a_t)$, a state transition function $\mathcal{P}(s_{t+1} \mid s_t, a_t)$, and the discount factor $\gamma$ for calculating the expected discounted return. An agent's behavior is defined by a policy, $\pi$, which maps states to a probability distribution over the actions $\pi : \mathcal{S} \rightarrow \mathcal{P}(\mathcal{A})$. The return from a state is defined as the sum of discounted future rewards $\mathcal{R}_t = \sum_{i=t}^{T} \gamma^{(i-t)} r(s_i, a_i)$. The goal of reinforcement learning is to learn an optimal policy that instructs the agent on the best action to take in each state to maximize its long-term reward.

In this study, our goal is to use RL to train the manipulator to plan a trajectory to reach an end-effector pose in the 3D space. Consider the environment where we want to control

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on November 01,2026 at 04:20:47 UTC from IEEE Xplore. Restrictions apply.

a 6-DOF manipulator's end-effector move towards an end pose $\chi_e$ in a 3D space. Where $\chi_e = [p_e,\ \phi_e]^T \in \mathcal{C}^7$ is a pose, with $p_e$ being the three-dimensional position and $\phi_e$ the orientation (e.g., quaternion) in the Cartesian space. The joint angles $\mathcal{J} = (J_1, J_2, J_3, J_4, J_5, J_6)$ denotes the angle of each joint on the manipulator in radians. Thus, we can calculate the pose of the end-effector $\chi_t$ at any given time $t$ using forward kinematics: $\chi_t = Forward_Kinematics\,(\mathcal{J})$. Similar with the end-effector pose $\chi_e$, the goal pose can be defined as $\chi_g = \begin{bmatrix} p_g, & \phi_g \end{bmatrix}^T \in \mathcal{C}^7$. The State of our model is the tuple of manipulator's current end-effector pose, joint angles and the goal pose $\mathcal{S} =< \chi_e, \mathcal{J}, \chi_g >$. Since we can only control the joints of the manipulator to achieved desired pose, the Action is defined as the angular speed of each joint $\mathcal{A} = \dot{\mathcal{J}} = \left(\dot{J}_1, \dot{J}_2, \dot{J}_3, \dot{J}_4, \dot{J}_5, \dot{J}_6\right)$.

Reward design is the most important part in this work. Since many previous research have worked on manipulator RL, our State and Action design is very similar to previous works. However, our Reward design is novel and unique since not many papers worked on the constrained trajectory planning before. Different from works like [17] [18], we use quaternion instead of eular angle to represent the rotation of end-effector and goal. Our aim is to match the end-effector pose $\chi_e$ from the starting pose $\chi_s$ to the goal pose $\chi_g$ by controlling $\dot{\mathcal{J}}$. We define the reward signal consists of four parts: positional reward, rotational reward, action reward and collision reward.

The first part of the reward function is designed for encouraging the end-effector approach a specific target position $p_e$. We utilize a nonlinear piecewise Eq.(1) to compute this part of the reward. In the formula, $d_{tar}$ is the Euclidean distance between the end-effector and its target position, the parameter $\delta$ is the turning point of the function. When the distance $d_{tar}$ is smaller than $\delta$, the reward function has a smaller slope for the agent to learn finer actions.

$$r_1 = \begin{cases} -\frac{1}{2} d_{tar}^2 & d_{tar} \leq \delta \\ -\delta \left( \left| d_{tar} \right| - \frac{1}{2}\delta \right) & d_{tar} > \delta \end{cases} \tag{1}$$

The second part is designed to encourage the end-effector arrive a specific target orientation $\phi_e$. As we mentioned previously, the rotation of the end-effector and the goal are represented using quaternions $\phi_e$ and $\phi_g$. Eq.(2) is used to calculate the angular distance between two quaternions.

$$r_2 = -2 \cdot \arccos(|\phi_e \cdot \phi_g|) \tag{2}$$

The third part is relative to the norm of the current action $\dot{J}_t$, which means we encourage the agent to take smaller actions for safety. This is shown in Eq.(3).

$$r_3 = -\left\| \dot{J}_t \right\| \tag{3}$$

The fourth part is designed for the purpose of avoiding collision, including collision with the table and the manipulator itself. In this part, we simply penalise the agent if any link on the manipulator collides with anything.

$$r_4 = \begin{cases} -1 & if\ Collision \\ 0 & Otherwise \end{cases} \tag{4}$$

Finally, the total reward signal is the weighted summation of (1),(2),(3),and (4). In Eq.(5), $\lambda_1$, $\lambda_2$, $\lambda_3$, $\lambda_4$ are the coefficients or weights of each part of the reward. In our model, we chose $\lambda_1$ to be 14, $\lambda_2$ to be 6, $\lambda_3$ to be 1 and $\lambda_4$ to be 500. These numbers are chosen based on our simulation experiments shown in Fig.3, especially for $\lambda_1$ and $\lambda_2$, the proportion of these two weights largely affects the training result of our model. The selection process of weights is further introduced in Section IV.

$$r_t\,(s_t, a_t) = \ \lambda_1 r_1 + \lambda_2 r_2 + \lambda_3 r_3 + \lambda_4 r_4 \tag{5}$$

### *B. Soft Actor-Critic*

Soft Actor-Critic (SAC), an off-policy Deep Reinforcement Learning (DRL) algorithm presented in [6], has manifested exemplary efficiency across a multitude of continuous control operations. Given its proven efficacy, it is an optimal choice for controlling our manipulator system. As inferred from its nomenclature, SAC operates on the principle of actor-critic, bifurcating its primary mechanism into two essential segments. The 'actor' segment focuses on formulating an effective policy for action selection, whereas the 'critic' section is responsible for appraising the value of distinct state-action configurations. The term 'soft' in SAC signifies the incorporation of entropy regularization within the learning process.

The concept of entropy is fundamentally a quantification of stochasticity or randomness within a system, with higher entropy corresponding to increased randomness. This notion holds significant relevance to the exploration-exploitation dichotomy, given that enhanced entropy directly correlates to heightened exploration. Distinguishing itself from alternative deep reinforcement learning algorithms such as DDPG and D4PG (Distributed Distributional Deterministic Policy Gradients), SAC promotes extensive exploration rather than merely pursuing optimal pathways during the exploration phase. This aspect significantly minimizes the risk of convergence to suboptimal local optima. In our research, the agent of interest is a 6-DoF manipulator operating within a three-dimensional realm, thus, an immensely vast exploration space. Consequently, the model requires a greater level of random action generation to facilitate comprehensive space exploration. In such conditions, SAC demonstrably surpasses the performance of its counterparts, thereby solidifying our selection of this algorithm for our study.

### *C. Hindsight Experience Replay*

HER is a technique introduced in [19] to enhance the learning process in reinforcement learning, particularly in tasks with sparse rewards, such as manipulator control. The core idea of HER is simple and intuitive: it reuses past failed experiences by altering the goals, creating more successful and positive experiences to accelerate the learning process. In constrained trajectory planning tasks, the reward is highest when the

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on November 01,2026 at 04:20:47 UTC from IEEE Xplore. Restrictions apply.

distance and orientation between the end-effector and goal are smaller than 1 cm and 5 degrees, respectively. This makes learning challenging for the agent, as the exploration space is vast and generating successful experiences is difficult. HER addresses this issue by saving the agent's past experiences in a replay buffer. Unlike a standard replay buffer, HER modifies these past experiences by changing the original failed goal to a successful goal and recalculating the reward based on the new goal. In this way, the agent can learn from generated positive trails and explore the environment more effectively. Algorithm 2 provides the pseudocode for HER.

## IV. SIMULATION

In the following section, we outline our robot setup within the simulated training environment and detail the resultant simulation experiment findings. A comparative analysis that examines our proposed reward function's effectiveness against alternative reward functions. Finally, we interpret and discuss the salient findings derived from our simulation results.

### *A. Simulation Setup*

In our endeavor to train the RL algorithm, we employed PyBullet, an open-source physics engine renowned for its utility in simulation, robotics, and machine learning, to construct a virtual environment mirroring our robot setup. The utilisation of this virtual environment not only expedites the training process beyond what is feasible with a real robot but also safeguards the robot against potential damages stemming from unpredictable movements during training. As depicted in Fig. 1, the simulated robot is meticulously mounted on a linear track atop the workbench, mirroring the actual setup to foster a seamless algorithm transfer in the future.

To enhance the visualisation of the goal position within the simulation environment, we employed a multicolored cube as a representative symbol. This cube, with its six faces painted in distinct colors and its front face colored in blue, offers a clear visual aid. Instances of successful goal attainment, where the end-effector perfectly aligns with the cube's center, are presented in Fig.2. This figure also delineates the 3D space accommodating the goal, signified by the transparent cube. The goal area, though significantly smaller than the UR5e manipulator's operational envelope, is carefully calibrated to ensure that the sampled goal can be reached from any random orientation without any collision.

As previously outlined in this paper, our model training capitalises on the vanilla SAC and HER algorithm, provided by Stable-Baselines3 [20], a library known for its stable and robust RL algorithm implementations. Certain hyperparameters were finetuned to optimise the training results, as listed in Table I.

### *B. Results*

In the pursuit of achieving optimal results with RL, one pivotal factor to be determined is the weighting accorded to both translational and orientational rewards. The weight on the reward essentially guides the degree of emphasis the algorithm will place upon it, and a balanced fifty percent distribution does not certainly yield the most favorable outcome. Hence, a simulation experiment was conducted to ascertain the optimal ratio of reward weight. Our simulation design sets the total weight at -20 and crafted nine evenly distributed scenarios ranging from -2 to -18. For each of these scenarios, the algorithm was trained for 1,000,000 epochs, maintaining all other parameters constant. The reward curves of these nine scenarios were subsequently illustrated in Fig.3. In the figure legend, each colored line signifies a specific weight setting. For instance, "T:-2, O:-18" displays the reward curve where the translational reward weight is -2, and the orientational reward weight is -18. After a thorough analysis of the presented data, it was concluded that -14 and -6 serve as the most suitable combination for translation and orientation reward, respectively. This result is predicated on the premise of achieving a harmonious balance between these two reward types, subsequently optimizing the efficiency and effectiveness of the RL process.

Fig.4 presents the three plots that represent the mean reward, success rate and epoch length, respectively. During the training phase, we test the model for 10 epochs after every 1,000 epochs of training. The mean reward, success rate and epoch length are calculated based on the 100 testing epochs, forming one testing checkpoint. Thus, out of 2 million training epochs,

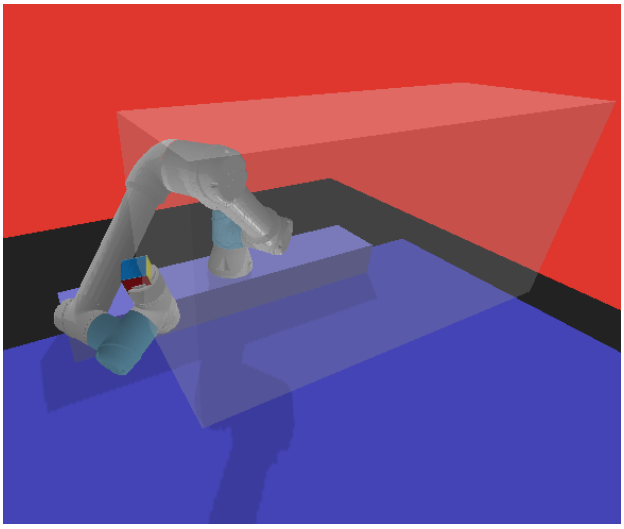


Fig. 2. The goal position and orientation are successfully achieved by the robot (robot tip overlaps with the coloured cube, pointing to the direction of the blue face), the goal sampling areais visualised by the transparent rectangular box

TABLE I
LIST OF HYPERPARAMETERS USED IN TRAINING

| Name | Value |
|---|---|
| Epochs | 2000000 |
| Steps | 100 |
| Buffer Size | 10000000 |
| Learning Rate | 0.0001 |
| Gamma | 0.95 |
| Batch Size | 256 |

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on November 01,2026 at 04:20:47 UTC from IEEE Xplore. Restrictions apply.

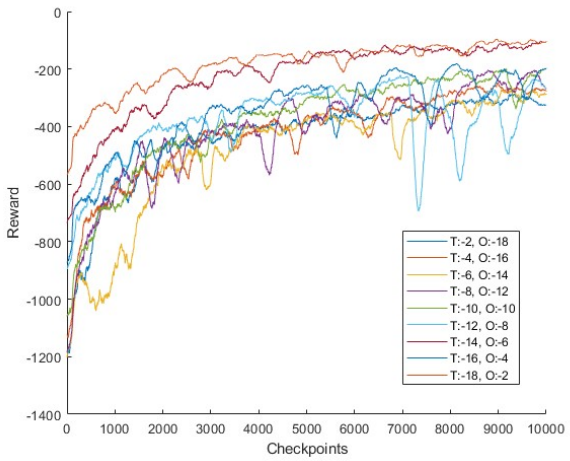



Fig. 3. Nine reward curves for different combinations of translational and orientational reward weight, the best combination will be selected as our reward hyperparameters

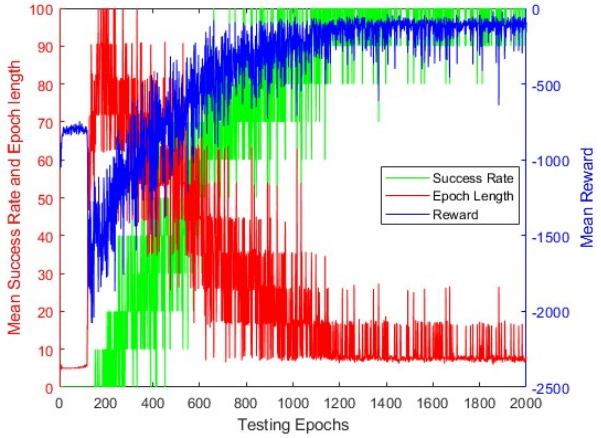



Fig. 4. The mean reward, success rate and epoch length calculated with 10 testing epochs after every 1,000 training epochs

we obtained 2,000 testing checkpoints. In Fig.4, the mean epoch length and success rate are presented following the left Y-axis, where the values for mean reward are presented on the right Y-axis. As shown in the figure, the reward initiates from a medium level. This is because the robot constantly collides with the table or itself during the very beginning of the learning. Consequently, the mean episode length remains very low, and the success rate is always zero. At around 150 checkpoints, the reward suddenly drops and subsequently continues to grow gradually, finally stabilizing around -50. This shows the robot is gradually learning how to achieve the goal. Consequently, the mean epoch length suddenly rises to 100 steps and then gradually drops, stabilising around 8 steps. This shows the robot is learning how to achieve the goal with as little time as possible. As a result of learning, the mean success rate in testing epochs grows from zero and stabilises between 90% and 100%.

Even though we have testing epochs during the training, the size of it is limited to 10, and the goal position and orientation are totally random. It cannot represent the real performance of our trained model. In response, we devised a test set to evaluate the trained model. As shown in Fig.2, the target pose is sampled within a cubic area, possessing dimensions of 0.8m, 0.3m, and 0.4m in length, width, and height, respectively. We implemented a systematic sampling strategy wherein a target position is sampled every 0.05m within the goal area, and for each of these target positions, five random orientations are sampled. This methodology resulted in a total of 5355 points (1779*5) within our test set. Upon evaluation of our trained model against this test set, we secured a success rate of 97%. This rate exemplifies the proficiency and reliability of our RL-based model within the specified parameters and dimensions.

In order to substantiate the efficacy of our reward engineering, we juxtaposed our reward design against two other reward functions presented in the literature, both addressing similar problems. The first reward function, as posited by Lin et al. (2021) [17], computes the reward directly by considering the translational distance and the product of the cosine of the orientational distance in each term of the rotation vector. This function is represented in Eq.(6). The second reward function, proposed by Cao et al. (2023) [18], employs the hyperbolic tangent of the translational distance and the cosine of the maximum orientational distance in the rotation vector to calculate the reward. This formula is detailed in Eq.(7). In these formulas, $e_r, e_p, e_y$ denote the roll, yaw, and pitch angles respectively, with reference to the discrepancy between the end-effector's orientation and the goal orientation.

$$r_t\left(s_t, a_t\right) = d_{tar} + 50 * cos\left(e_r\right) * cos\left(e_p\right) * cos\left(e_y\right) \quad (6)$$

$$r_t\left(s_t, a_t\right) = 1 - tanh\left(d_{tar}\right) + cos\left(max[e_r, e_p, e_y]\right) \quad (7)$$

Fig.5 illustrates the reward curve generated via our proposed reward function, in comparison to reward function one and reward function two. From the graph, it becomes apparent that our reward remains consistently superior compared to the other two. However, it is critical to underscore that the original reward curves cannot be directly compared, given that the maximum and minimum rewards are distinct. To facilitate a more comprehensible visual understanding, we have normalized the three curves to the same scale. As previously noted, the reward curve primarily depicts the learning trend and does not provide a definitive measure of the trained model's performance. Consequently, we subjected models trained with Eq.(6) and Eq.(7) to testing on our established test set. The results revealed a success rate of 50% for the model trained with reward function one, whereas the model trained with reward function two exhibited a success rate of 20%.

### C. Discussion

In the preceding section, a comparative analysis was conducted between our reward design and two other designs,

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on November 01,2026 at 04:20:47 UTC from IEEE Xplore. Restrictions apply.

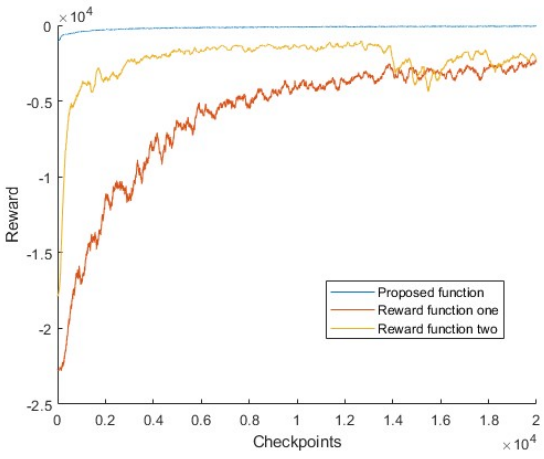



Fig. 5. Comparing the reward curve between our reward design with the other designs

with the success rate demonstrating significant disparity. The rationale behind these results can be attributed to the training methodologies and problem modeling utilized in the three approaches. In the study [17] wherein Eq.(6) was proposed, Recurrent DDPG was deployed, and the problem was simplified by utilising a 3-DOF manipulator instead of a 6-DOF one. The research [18] proposing Eq.(7) employed inverse kinematics as prior knowledge to support the training. Thus, performance degradation was observed when a vanilla SAC algorithm was implemented. In contrast, our reward function employs a quaternion to represent the rotation angle instead of a rotation vector, thereby circumventing potential issues such as "gimbal lock" that may be encountered with rotation vectors. This allows the robot to learn trajectory planning and control more accurately even with the vanilla SAC algorithm.

## V. CONCLUSIONS AND FUTURE WORK

In conclusion, we have delved into the potential of RL in trajectory planning under constraints, which aiming to let robotic manipulators learn to plan and execute a trajectory and achieve the designated target TCP 6D pose. The innovative approach to our reward function design was also explored. Our design considers quaternion representation for rotation, which effectively avoids "gimbal lock" issues, thereby enabling the robot to learn trajectory planning and control with greater precision. Comparative analysis with other reward functions has demonstrated our design's superiority, providing a significantly higher success rate.

In line with our observations, future research directions should focus on the development of reinforcement learning algorithms that tackles trajectory planning problem with moving obstacles in 3D space. Further studies could also explore how best to integrate RL algorithms with other AI technologies to compose a Hierarchical Reinforcement Learning system, achieving more intelligent and flexible robotic manipulators for the smart manufacturing industry.

## REFERENCES


[1] Y. Lu, X. Xu, and L. Wang, "Smart manufacturing process and system automation–a critical review of the standards and envisioned scenarios," *Journal of Manufacturing Systems*, vol. 56, pp. 312–325, 2020.

[2] Y. Lee, E. S. Hu, and J. J. Lim, "Ikea furniture assembly environment for long-horizon complex manipulation tasks," in *2021 ieee international conference on robotics and automation (icra)*. IEEE, 2021, pp. 6343–6349.

[3] S. Levine, P. Pastor, A. Krizhevsky, J. Ibarz, and D. Quillen, "Learning hand-eye coordination for robotic grasping with deep learning and large-scale data collection," *The International journal of robotics research*, vol. 37, no. 4-5, pp. 421–436, 2018.

[4] S. Gu, E. Holly, T. Lillicrap, and S. Levine, "Deep reinforcement learning for robotic manipulation with asynchronous off-policy updates," in *2017 IEEE international conference on robotics and automation (ICRA)*. IEEE, 2017, pp. 3389–3396.

[5] J. Zhang, W. Zhang, R. Song, L. Ma, and Y. Li, "Grasp for stacking via deep reinforcement learning," in *2020 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2020, pp. 2543–2549.

[6] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *International conference on machine learning*. PMLR, 2018, pp. 1861–1870.

[7] J. Ibarz, J. Tan, C. Finn, M. Kalakrishnan, P. Pastor, and S. Levine, "How to train your robot with deep reinforcement learning: lessons we have learned," *The International Journal of Robotics Research*, vol. 40, no. 4-5, pp. 698–721, 2021.

[8] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra, "Continuous control with deep reinforcement learning," *arXiv preprint arXiv:1509.02971*, 2015.

[9] S. Gu, T. Lillicrap, I. Sutskever, and S. Levine, "Continuous deep q-learning with model-based acceleration," in *International conference on machine learning*. PMLR, 2016, pp. 2829–2838.

[10] J. Schulman, S. Levine, P. Abbeel, M. Jordan, and P. Moritz, "Trust region policy optimization," in *International conference on machine learning*. PMLR, 2015, pp. 1889–1897.

[11] S. M. Kakade, "A natural policy gradient," *Advances in neural information processing systems*, vol. 14, 2001.

[12] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[13] S. Wang, X. Zheng, Y. Cao, and T. Zhang, "A multi-target trajectory planning of a 6-dof free-floating space robot via reinforcement learning," in *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2021, pp. 3724–3730.

[14] J. Zhong, T. Wang, and L. Cheng, "Collision-free path planning for welding manipulator via hybrid algorithm of deep reinforcement learning and inverse kinematics," *Complex & Intelligent Systems*, pp. 1–14, 2021.

[15] Y. Li, X. Hao, Y. She, S. Li, and M. Yu, "Constrained motion planning of free-float dual-arm space manipulator via deep reinforcement learning," *Aerospace Science and Technology*, vol. 109, p. 106446, 2021.

[16] Y. Li, D. Li, W. Zhu, J. Sun, X. Zhang, and S. Li, "Constrained motion planning of 7-dof space manipulator via deep reinforcement learning combined with artificial potential field," *Aerospace*, vol. 9, no. 3, p. 163, 2022.

[17] G. Lin, L. Zhu, J. Li, X. Zou, and Y. Tang, "Collision-free path planning for a guava-harvesting robot based on recurrent deep reinforcement learning," *Computers and Electronics in Agriculture*, vol. 188, p. 106350, 2021.

[18] Y. Cao, S. Wang, X. Zheng, W. Ma, X. Xie, and L. Liu, "Reinforcement learning with prior policy guidance for motion planning of dual-arm free-floating space robot," *Aerospace Science and Technology*, p. 108098, 2023.

[19] M. Andrychowicz, F. Wolski, A. Ray, J. Schneider, R. Fong, P. Welinder, B. McGrew, J. Tobin, O. Pieter Abbeel, and W. Zaremba, "Hindsight experience replay," *Advances in neural information processing systems*, vol. 30, 2017.

[20] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, "Stable-baselines3: Reliable reinforcement learning implementations," *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021. [Online]. Available: http://jmlr.org/papers/v22/20-1364.html


Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on November 01,2026 at 04:20:47 UTC from IEEE Xplore. Restrictions apply.